CHAPTER 1

DESIGNING THE AMAZON LOCKER SERVICE

1 Getting Ready: Amazon Locker Service 2
2 Requirements for the Amazon Locker Service 5
3 Use Case Diagram for the Amazon Locker Service . . . 6
4 Class Diagram for the Amazon Locker Service 9
5 Sequence Diagram for the Amazon Locker Service . . . 21
6 Activity Diagram for the Amazon Locker Service . . . 23
7 Code for the Amazon Locker Service 24
8 Summary: Designing the Amazon Locker Service . . . 34
9 Key Insights and Tips: Designing the Amazon Locker Service 37

This chapter covers the design of the Amazon Locker Service, focusing on requirements, UML diagrams, and implementation code to create a complete system.

1 Getting Ready: Amazon Locker Service

1.1 Problem definition

Amazon is an online retailer that allows customers to order products for delivery. Sometimes, customers are unavailable at their usual address to receive packages. In such cases, Amazon provides a secure, automated pickup and return service called **Amazon Locker**.
Amazon Lockers are available at various locations, each containing multiple lockers of different sizes and operating hours. Customers can select a nearby locker location for delivery or returns. Only packages that fit a locker's size are eligible. When a package arrives, the customer receives a unique code to retrieve it. Packages are stored for a limited period; if uncollected, they are removed, and the customer may be refunded. Customers can also return eligible products by dropping them off at available locker locations, where the logistics team picks them up.
This scenario presents interesting challenges: real-time resource allocation, handling concurrent user requests, operating hour constraints, managing physical locker and package sizes, and supporting delivery and return flows.
In this LLD interview case study, you'll focus on:

- Assigning lockers to orders and returns based on size and availability.
- Handling package pickups, returns, and uncollected items.
- Managing locker states, codes, and access windows.
- Ensuring the system works reliably and securely at scale.

This system model can be adapted for any retailer providing automated, secure delivery and returns.

Figure 1.1: The Amazon Locker service

1.2 Expectations from the interviewee

The Amazon Locker service consists of multiple components, each with functionality and constraints. The following section provides an overview of some of the main expectations the interviewer will want to hear you discuss in more detail during the interview.
In the interview, you are expected to:

- Clarify system requirements, constraints, and edge cases by asking relevant questions.
- Identify and design core system components.
- Justify design decisions using OOD and SOLID principles.

Typical areas you should probe and discuss include:

Locker size

Every locker is of a specific size in the Amazon Locker system. The interviewer expects you to ask the questions listed below:

- Are there different locker sizes? How does package size affect locker assignment?
- Are there restrictions on what can be stored in each locker?

Locker selection

The most significant part of the Amazon Locker service is the selection of the locker. The system has to ensure that more than one customer cannot access a locker at the same time. The interviewer expects you to ask the questions listed below to identify how the system will work in such situations:

- How will the system ensure multiple customers do not get the same locker?
- Will the customer choose the locker of his own choice, or will the system assign him a locker based on availability?
- Can a customer get two lockers for different orders at the same time?
- Will the system consider the locker and package sizes while assigning the locker to the customer?

Locker status

As this problem revolves around the locker, you may ask the questions listed below about the locker status:

- Is there any time constraint on the package that can be kept in the locker?
- What will happen if the customer does not come to pick up his package within the valid time period?

Returning an item

Like the order delivery process, the item can be returned through the Amazon Locker service. Therefore, you may ask the questions listed below:

- Can the customer return an item through the Amazon Locker service?
- If yes, will they get the same locker from which they picked the item?
- How will the locker be assigned to the customer while returning an item?

1.3 Design approach

We will design this Amazon Locker service using the bottom-up design approach. For this purpose, we will follow the steps below:

- First, we'll identify core entities like `Locker`, `LockerLocation`, `Package`, and `Order`, and define their responsibilities.
- Next, we'll model how lockers are assigned to packages based on size and availability, and handle package pickups, returns, and uncollected items.
- We'll ensure that locker operations respect time limits and location operating hours and support secure access via unique codes.

This design approach will address concurrency and edge cases and follow SOLID principles to ensure scalability and maintainability. Later , diagrams and code will illustrate major workflows and class structures.

1.4 Design pattern

During an interview, discussing the design patterns under which the Amazon Locker system falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking "Define design patterns."

Question:

Which design pattern(s) should be used to design an Amazon Locker service? Kindly elaborate on your choice(s).

Let's explore the requirements of the Amazon Locker service in the next lesson.

2 Requirements for the Amazon Locker Service

This lesson outlines the functional and operational requirements for the Amazon Locker service. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

The requirements for the Amazon Locker service are defined below:

- **R1:** Customers can select a preferred locker location for order pickup during checkout.

Column component type 'MxGraphWidget' not yet supported.

- **R2:** An order may contain one or more items. Based on locker size availability, items are packaged together if possible.
- **R3:** Locker locations contain multiple lockers of various sizes (extra small, small, medium, large, extra large, double extra large).
- **R4:** Only packages that fit fully within the locker's interior dimensions are eligible for delivery.
- **R5:** When a package is delivered to the selected locker, the customer receives a unique code (e.g., a 6-digit PIN) to open the locker.

Column component type 'MxGraphWidget' not yet supported.

- **R6:** Packages are held in the locker for three days.
- **R7:** Each locker location has defined opening and closing hours; customers must pick up packages within the 3-day window and the location's operating hours.

- **R8:** If a package is not picked up within three days, it is removed from the locker, the locker is released, and the customer is refunded.

Column component type 'MxGraphWidget' not yet supported.

- **R9:** Multiple lockers are available at every locker location; each can only be assigned to one customer/package at a time.
- **R10:** Once a package is collected, the locker is closed and locked; the provided access code is invalidated and cannot be reused.
- **R11:** Customers may return eligible items by selecting a nearby locker location. Based on package size and location, an available locker is assigned. A new, unique code is sent to the user to open the locker and place the return package.

Column component type 'MxGraphWidget' not yet supported.

- **R12:** The logistics team stores returned items in lockers for pickup; the logistics team uses a unique code to collect the returned package. The customer is notified once the return is processed, and the refund policy is applied per product eligibility.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases for the Amazon Locker system.

3 Use Case Diagram for the Amazon Locker Service

Let's build the use case diagram of the Amazon Locker System and understand the relationship between its main actors and functions. First , we'll define the different elements of our system, followed by the complete use case diagram.

3.1 System

Our system is the Amazon Locker system. It manages automated package deliveries and returns via secure locker locations.

3.2 Actors

Now, we'll define the main actors of our Amazon Locker system.

Primary actors

- **Customer:** This Amazon customer ordered a package delivered to the Amazon Locker. The customer is responsible for selecting and booking a locker location for their order delivery. This actor can enter the code at the locker to retrieve their product, request a return, and drop off the package.

- **Delivery person:** This actor can also enter the code and add the product to the locker so the "Customer" can pick it up. This actor can also pick up a returned package from the locker. The delivery person uses unique, one-time codes for each delivery or return.

3.3 Use cases

This section defines locker use cases. We have listed them according to their respective interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Customer

- **Select locker location:** Tochoose a preferred location for package delivery or product return.

- **Pick up package:** Toretrieve the delivered package from the locker after unlocking it.

- **Remove package:** To collect a package from the locker after delivery or for return.

- **Add package:** To use the provided access code to place a return package inside the locker.

- **Submit return request:** To start the process to return a purchased product through a locker.

- **Receive delivery notification:** To get notified when a package has been delivered to the assigned locker, including locker details and access code.

- **Receive return notification:** To get notified with instructions and code for returning a product via a locker.

- **Receive overdue notification:** To get notified if the package was not picked up before the deadline.

Delivery person

- **Deliver package:** To place a customer's package inside the assigned locker for pickup.
- **Remove package:** To retrieve returned items from the locker for processing or transport.
- **Receive return notification:** To get alerted about packages that have been returned and are ready for pickup.

System

- **Issue locker:** To assign a suitable locker to a package based on size and availability.
- **Generate code:** To create a unique access code for locker entry.
- **Validate code:** To check if the entered code is correct and valid for locker access.
- **Find locker:** To locate the correct locker corresponding to the access code.
- **Lock/unlock locker door:** To control the mechanism to secure or open the locker door.
- **Send delivery notification:** To inform the customer that their package has been delivered and provide locker details and code.
- **Send return notification:** To notify the delivery person about returned products that require pickup.
- **Send overdue notification:** To alert the customer when a package hasn't been collected within the specified time frame.

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Customer	**Delivery Person**	**Amazon Locker System**
Select locker location	Deliver package	Issue locker
Pick up package	Remove package	Generate code
Submit return request	Add package	Remove package
Find locker	Lock/unlock locker door	Validate code
Receive delivery notification		Receive return notification
		Send delivery notification
		Send return notification
		Receive overdue notification
		Send overdue notification

Include

When a customer picks up, adds, or removes a product from a locker, or when a delivery person delivers a package, they must first enter a code. The system checks the code's validity, locates the appropriate locker, and unlocks the door. These are best represented using "include" relationships in a use case diagram:

- "Pick up package," "Add package," "Deliver package," and "Remove package" use cases all include the "Enter code" use case.
- "Enter code" includes "Validate code" use case.
- "Validate code" includes "Find locker."
- "Find locker" includes "Lock/unlock locker door."

For product returns, the customer initiates the process on the Amazon Locker system by submitting a return request. After the request is approved, the Amazon Locker system sends a return notification and generates a unique code for locker access.

- The " Submit return request"use case includes the"Send return notification"use case.
- The " Send return notification"use case includes the"Generate code"use case.

The system generates the required access code when a locker is assigned for an order.

- The "Issue locker" use case includes the "Generate code" use case.

3.5 Use case diagram

Here's the use case diagram for the Amazon Locker system:

In the next lesson, we will discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Amazon Locker Service

We'll create the class diagram for the Amazon Locker service. In the class diagram, we will first design the system's classes and then identify the relationship between classes according to the Amazon Locker service problem requirements.

4.1 Components of an Amazon Locker service

In this section, we'll define the classes for an Amazon Locker service. As mentioned, we are following the bottom-up approach to design a class diagram for the Amazon Locker service.

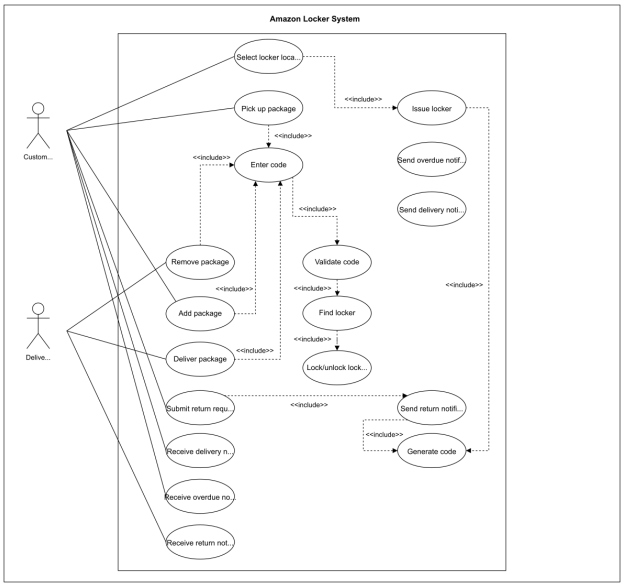
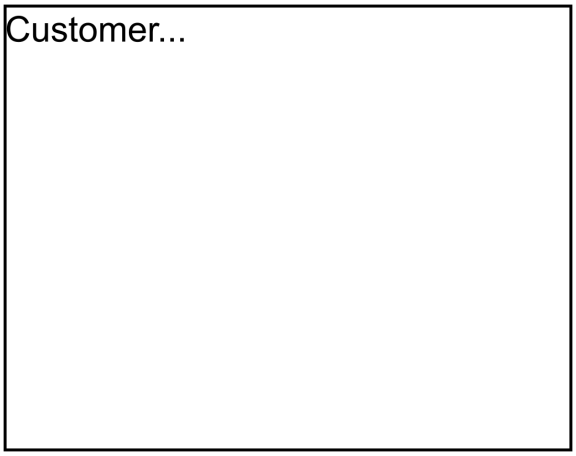


Figure 1.2: The use case diagram for the Amazon Locker system

Customer

The `Customer` class models an Amazon user interacting with the locker system to place orders, initiate returns, and receive notifications. It holds identifying and contact information, and provides methods to request orders and returns, supporting all customer-facing workflows.

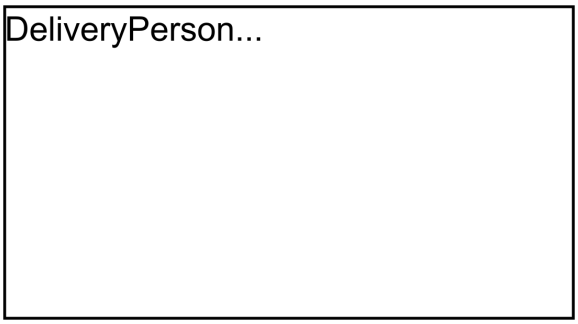


R1: A customer can select a preferred locker location for order pickup during checkout.
R2: An order may contain one or more items. Based on locker size availability, items are packaged together if possible.
R11: Customers may return eligible items by selecting a nearby locker location. An available locker is assigned based on package size and location, and a new unique code is sent to the user to open the locker and place the return package.
R12: Returned items are stored in lockers for pickup by the logistics team. The team uses

a unique code to collect the returned package. The customer is notified once the return is processed, and the refund policy is applied as per product eligibility.

DeliveryPerson

The `DeliveryPerson` class models the logistics staff member who delivers packages to lockers and collects returned items. It uses a unique identifier for each delivery person and provides methods to perform deliveries, pick up returns, and process return notifications. Personal contact information is not stored, keeping the class simple and focused on system operations.



R11: Customers may return eligible items by selecting a nearby locker location. An available locker is assigned based on package size and location, and a new unique code is sent to the user to open the locker and place the return package.
R12: Returned items are stored in lockers for pickup by the logistics team. The team uses a unique code to collect the returned package. The customer is notified once the return is processed, and the refund policy is applied as per product eligibility.

Item

The `Item` class represents an individual product that can be part of a customer's order. Each item has a unique identifier (`itemId`), a descriptive name, and a quantity specifying how many units are included in the order. This class allows the system to keep track of each product purchased, whether for delivery or return, and supports the packaging and order fulfillment processes.
The class representation is shown below:
R2: An order may contain one or more items. Based on locker size availability, items are packaged together if possible.

Order

The `Order` class models a complete transaction initiated by a customer. It includes a unique order identifier (`orderId`), a reference to the customer (`customerId`), a list of items (`items`), and a delivery location, which is typically an instance of `LockerLocation`. Orders are the starting point for delivery and return workflows and serve as the central record for tracking the status of purchased products.

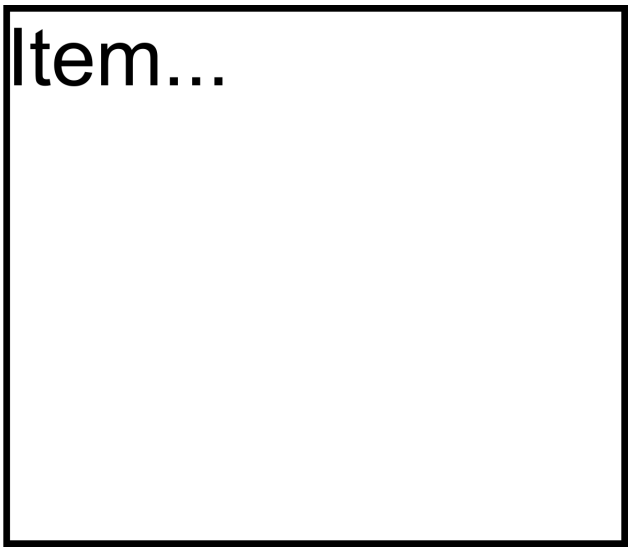


Figure 1.3: The class diagram of the Item class

The UML representation of a class is shown below:

Figure 1.4: The class diagram of the Order class

R1: A customer can select a preferred locker location for order pickup during checkout.
R2: An order may contain one or more items. Based on locker size availability, items are packaged together if possible.
R12: Returned items are stored in lockers for pickup by the logistics team. The customer is notified once the return is processed, and the refund policy is applied according to product eligibility.

Notification

The `Notification` class manages communication with customers and delivery personnel. Each notification includes a recipient, order, and locker references, a code if needed, type (delivery, return, overdue), and time stamp.
The class definition is given below:

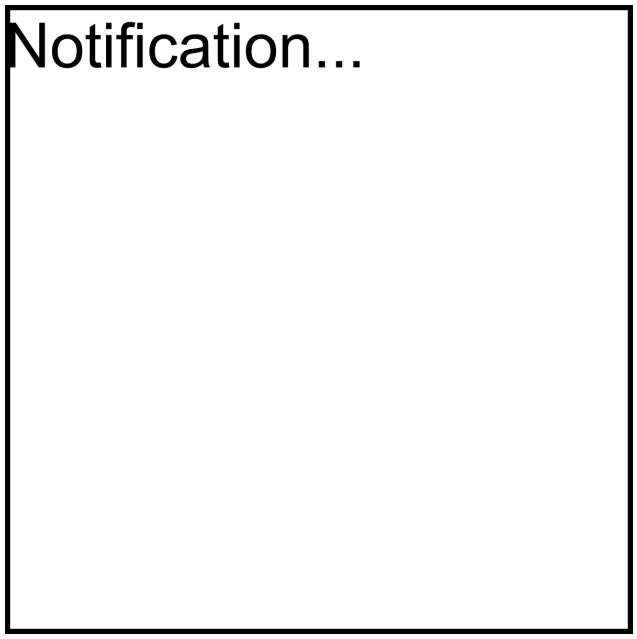


Figure 1.5: The class diagram of the Notification class

R5: When a package is delivered to the selected locker, the customer receives a unique code (e.g., a 6-digit PIN) to open the locker.
R7: Each locker location has defined opening and closing hours; customers must pick up packages within the 3-day window and location's operating hours.
R11: The logistics team stores returned items in lockers for pickup; the customer is notified once the return is processed, and the refund policy is applied as per product eligibility.

Package and locker package

The `Package` class encapsulates the physical bundle prepared for delivery or return. Each package has a unique `packageId`, a `size` attribute reflecting the locker size it can fit in, and a reference to the corresponding `Order`. An `isReturn` flag indicates whether the package is outbound for delivery or inbound as a customer return.
The `LockerPackage` class extends `Package` by adding information specific to the package's assignment to a locker. It includes the `lockerId`, a unique `code` for secure access, the `packageDeliverTime`, and `codeValidDays` to manage how long the access code is valid.
The representation of the `Package` and `LockerPackage` classes is shown below:

R4: Only packages that fit fully within the interior dimensions of a locker are eligible for locker delivery.
R5: When a package is delivered to the selected locker, the customer receives a unique code (e.g., a 6-digit PIN) to open the locker.
R6: Packages are held in the locker for three days.
R7: Each locker location has defined opening and closing hours; customers must pick up packages within the 3-day window and location's operating hours.
R8: If a package is not picked up within three days, it is removed from the locker, the locker is released, and the refund process is initiated for the customer.

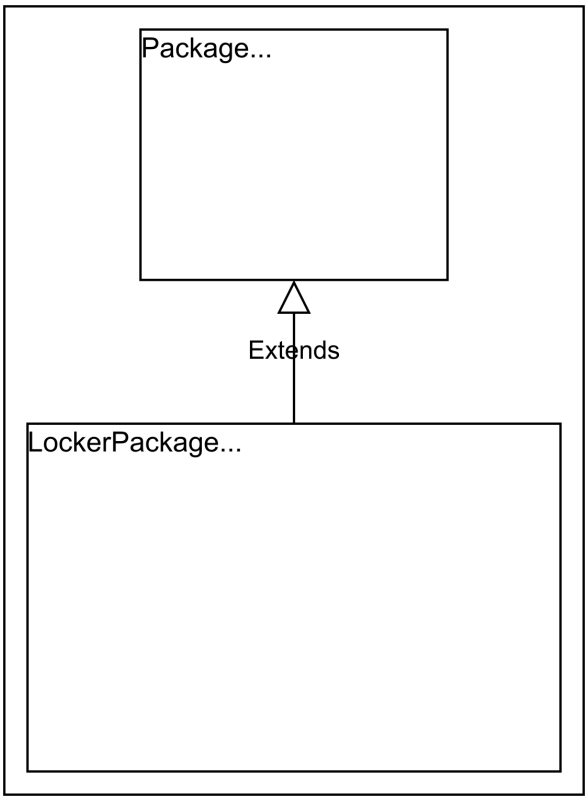


Figure 1.6: The class diagram of the Package and LockerPackage classes

Locker

The `Locker` class represents a physical compartment at a locker location. Each locker has a unique `lockerId`, a size, a reference to its `locationId`, and a status that indicates if it is available, booked, or occupied. The locker can also reference the current package it holds, if any.
The UML representation of a class is shown below:

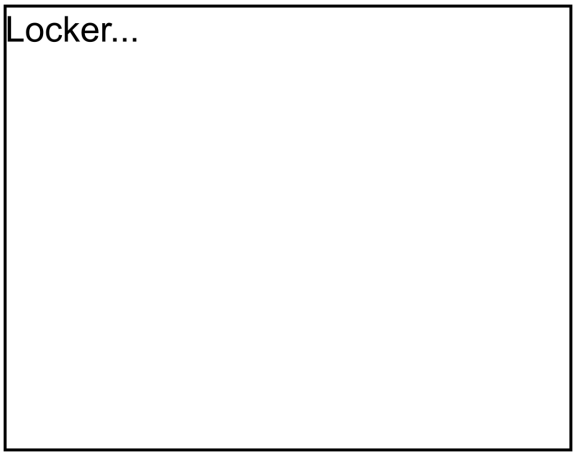


Figure 1.7: The class diagram of the Locker class

R3: Locker locations contain multiple lockers of various sizes (extra small, small, medium, large, extra large, double extra large).
R9: Multiple lockers are available at every locker location; each can only be assigned to one customer/package at a time.
R10: Once a package is collected, the locker is closed and locked; the provided access code is invalidated and cannot be reused.

Locker location

The `LockerLocation` class models a physical site with multiple lockers. It has a unique `locationId`, geographic coordinates, operational hours, and a list of `Locker` objects at that site.
The representation of this class is given below:
R1: A customer can select a preferred locker location for order pickup during checkout.
R9: Multiple lockers are available at every locker location; each can only be assigned to one customer/package at a time.
R10: Once a package is collected, the locker is closed and locked; the provided access code is invalidated and cannot be reused.

Locker service

The `LockerService` class is the main class of the Amazon Locker service system and contains a reference to the list of locker locations.

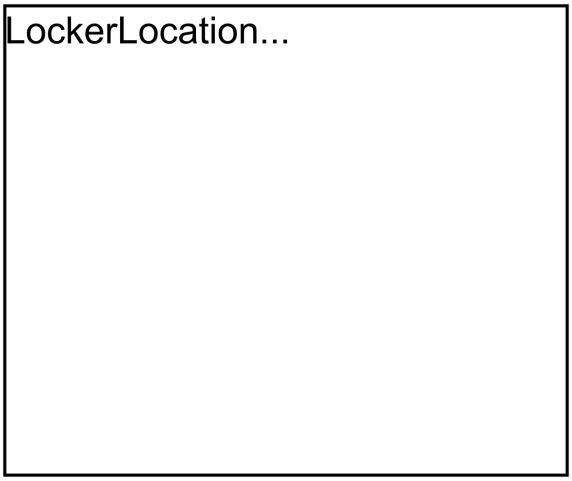


Figure 1.8: The class diagram of the LockerLocation class

The UML representation of the class is given below:

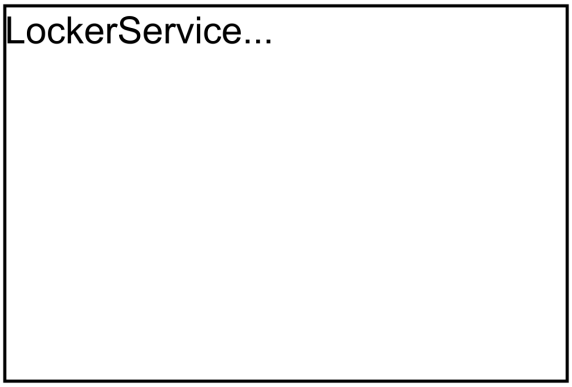


Figure 1.9: The class diagram of the LockerService class

R1: While ordering, customers can choose the nearest locker location from which to pick up their package.

Enumerations

The list of enumerations required in the Amazon Locker service is provided below:

- `LockerStatus`: The locker status describes the current status of the locker, whether it is closed, booked, or available.
- `LockerSize`: The locker size expresses the size of the locker, whether extra small, small, medium, large, extra large, or double extra large.

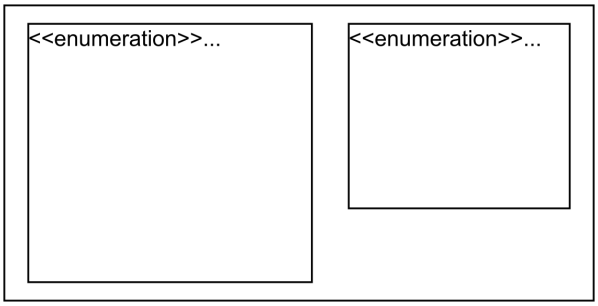


Figure 1.10: Enums in the Amazon Locker service

R3: Lockers can be different sizes, such as extra small, small, medium, large, extra large, and double extra large.
R13: When the customer picks up the order package from the locker, the locker's state is changed to closed, and the customer can no longer open the locker with the given code.

4.2 Relationship between the classes

Now, we'll discuss the relationships between the classes in the Amazon Locker service.

Association

The class diagram has the following association relationships:

- The `Notification` class has a one-way association with the `Locker` and `Order` classes, because notifications need to include details about the specific locker and order involved, but lockers and orders do not need to know about notifications.
- The `Notification` class is also associated with the `Customer` and `DeliveryPerson` classes, because notifications are sent directly to the user (customer or delivery person) involved in each event, while these user objects do not need to track their received notifications.
- The `Order` class has a one-way association with the `Customer` class because each order must be linked to the customer who placed it, but customers do not need to keep direct references to all their orders inside their own object.
- The `LockerPackage` class is associated with the `DeliveryPerson` class because when a package is delivered or picked up, it must be linked to the responsible delivery person, but the delivery person object does not need to store all package references.

Aggregation and composition

The class diagram has the following aggregation and composition relationships:

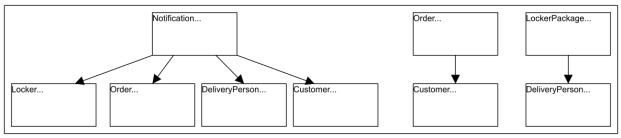


Figure 1.11: The association relationship between classes

- The `LockerService` class comprises the `LockerLocation` class, because `LockerService` manages and controls multiple locker locations, but each location can exist independently of the service.
- The `LockerLocation` class is an aggregation of the `Locker` class, as a locker location can have multiple lockers, which are managed collectively at a location but can exist independently.
- The `Locker` class is composed of the `LockerPackage` class, because each locker can contain a package, and when a locker is emptied, the association to that package ends.
- The `Package` class is aggregated of the `Order` class, which is composed of the `Item` class, because each package corresponds to an order (which may outlive the package), and each order is fundamentally made up of one or more items (which do not exist separately from the order).

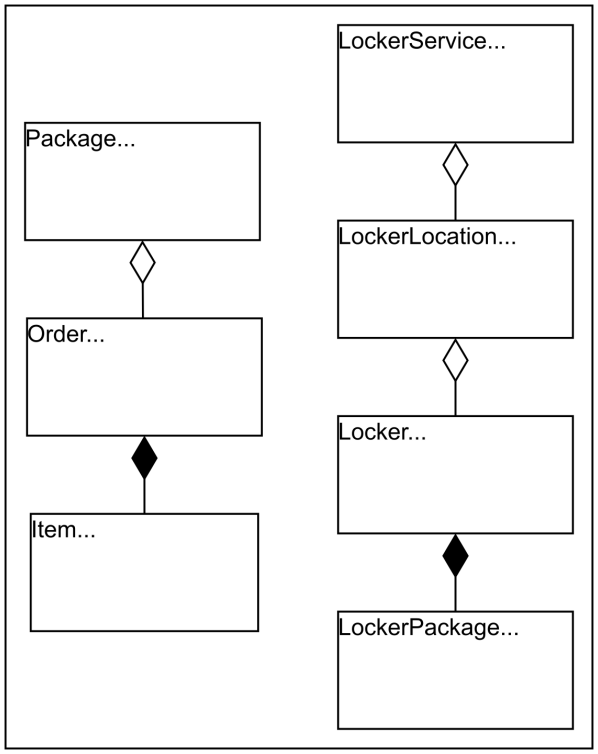


Figure 1.12: The composition relationship between classes

Inheritance

The following classes show an inheritance relationship:

- The `LockerPackage` class extends the `Package` class because a locker package is a special type of package with additional information specific to being stored in a locker.

  Note: The component section above has already discussed the inheritance relationship between classes.

4.3 Class diagram of the Amazon Locker service

This section outlines the multiplicity (cardinality) relationships between the main classes in our Amazon Locker system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	**Target**	**Multiplicity**	**Reason**
Order	Item	1 — 1..*	An order is made up of one or more items.
Order	Notification	1 — 1..*	Each order can generate many notifications.
Order	Package	1 — 1	Each order is packed into one package.
Package	LockerPackage	1 — 0..1 (extends)	Package may be represented as LockerPackage if put in locker.
Locker	LockerPackage	1 — 0..1	A locker can contain at most one package (LockerPackage).
LockerLocation	Locker	1 — 1..*	Each location has one or more lockers.
LockerService	LockerLocation	1 — 1..*	The system manages multiple locations.
Customer	Order	1 — 0..*	Customers can place many orders.
Customer	Notification	1 — 0..*	Customers may receive multiple notifications.
DeliveryPerson	Notification	1 — 0..*	Delivery persons can receive many notifications.
Notification	Order, Locker	1 — 1 (each)	Every notification relates to a specific order and locker.

Here's the complete class diagram for our Amazon Locker service:

4.4 Design pattern

The Amazon Locker service has multiple lockers at each locker location, and customers can choose their preferred location for package delivery or return. The Strategy design pattern can help assign the most appropriate locker, considering factors like customer location, locker size, and locker availability. By applying this pattern, the system can easily switch between

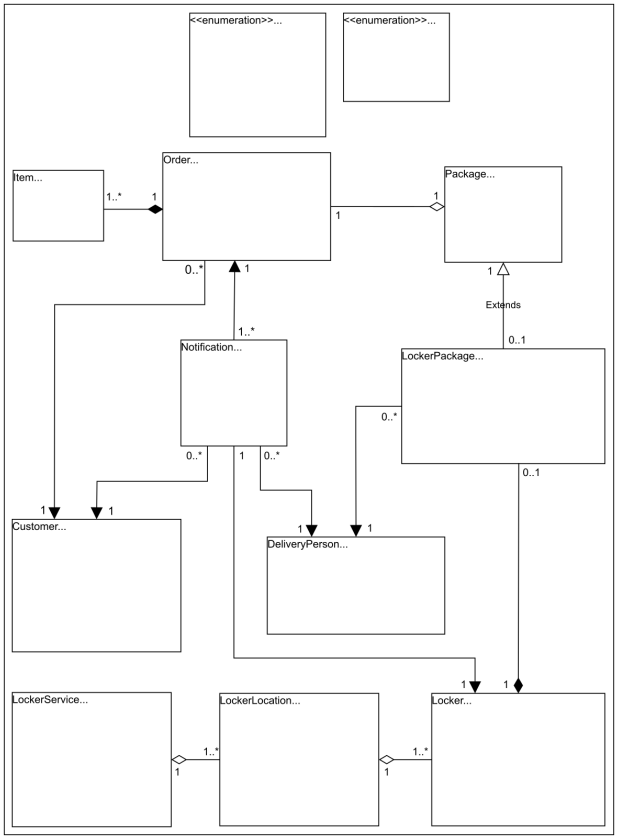


Figure 1.13: The class diagram of the Amazon Locker service

different locker assignment strategies (e.g., best fit by size, closest location, or load balancing), making the design flexible and extensible.
Other operations in the system can also leverage the Strategy pattern, such as:

- OTP (one-time password) generation algorithms
- Random number generation for codes
- Locker assignment and filtering logic

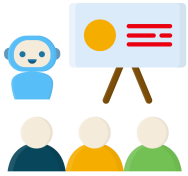
The Repository design pattern can be used to organize data access and promote a clean separation between business logic and data storage. For example:

- A `LockerRepository` can manage all lockers-related operations, such as searching, adding, or updating lockers.
- A `PackageRepository` can encapsulate package access, allowing for easy retrieval, storage, and updates of package data.

By applying these patterns, the Amazon Locker system becomes easier to maintain, test, and extend—key qualities for robust low-level design.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



We have completed the class diagram of the Amazon Locker service according to the requirements. In the next lesson, let's design its sequence diagram.

5 Sequence Diagram for the Amazon Locker Service

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. We can create different sequence diagrams for our Amazon Locker system. We will create a sequence diagram for returning a package in this lesson.

5.1 Return package

The sequence diagram for the package return should have the following actors and objects that will interact with each other:
Actors / Participants

- Customer (the person returning the package)
- LockerService (the central system that manages lockers and OTPs)
- Locker (an individual locker unit)
- Notification (responsible for sending OTPs and messages to the customer)

Here are the steps in the return package interaction:

1. The customer asks the system to initiate a package return.
2. System checks approval:
   (a) If the return is *denied*, the system immediately notifies the customer, and the process ends.
   (b) If the return is approved, proceed to the next step.
3. The system finds the first free locker and reserves it.
4. Send and verify OTP:
   (a) The system sends the customer a one-time passcode (OTP).
   (b) The customer submits the OTP back to the system.
   (c) If the code is *invalid or expired*, the system informs the customer and stops.
   (d) If the code is valid, move on to the next step.
5. Assign locker and place package:
   (a) The system assigns the reserved locker to the customer (marks it as booked).
   (b) The customer places the package into that locker, completing the return.
6. The process is finished. Once the package is in place, no further action is needed.

Based on the order above, the sequence diagram of package return in the Amazon Locker system is given below:

Next, let's examine the activity diagrams for the Amazon Locker system to understand its control flow.

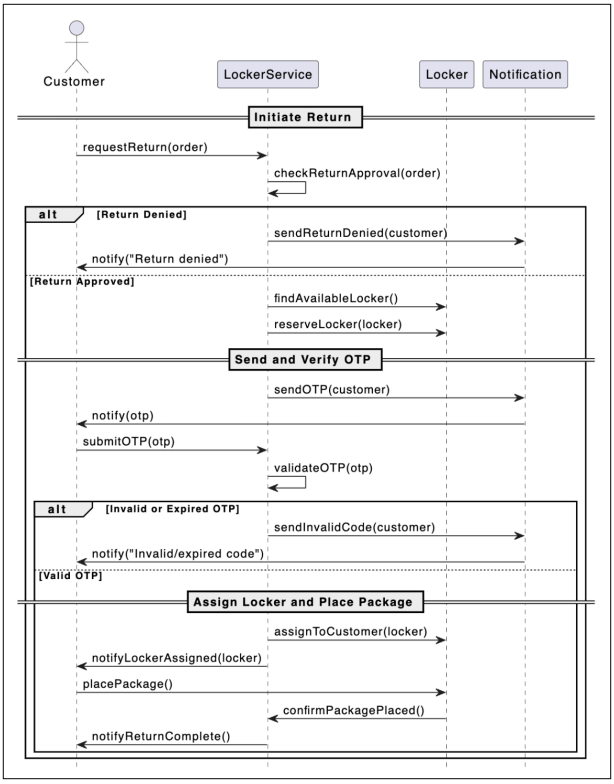


Figure 1.14: The sequence diagram for returning a package

6 Activity Diagram for the Amazon Locker Service

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our Amazon Locker system. In this lesson, we will create an activity diagram for the product pickup activity.

6.1 Product pickup

The states and actions involved in this activity diagram are provided below.

States

- **Initial state:** A customer who has ordered a product from Amazon comes to the Amazon Locker to pick it up.

- **Final state:** The customer successfully gets the product or the system shows an incorrect code error.

Actions

The customer arrives at the Amazon Locker and enters the code. The system validates the code and opens the locker.
Based on the order above, the activity diagram of product pickup is given below:

In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the Amazon Locker Service

We've reviewed the different aspects of the Amazon Locker service and observed the attributes attached to the problem using various UML diagrams. Now , let's explore the more practical side of things, where we will work on implementing the Amazon Locker service using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the Amazon Locker service:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Amazon Locker Service.

7.1 Amazon Locker service classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Enumerations

First, we will define all the enumerations used in the Amazon Locker service. According to the class diagram, the system has two enumerations, i.e., `LockerSize` and `LockerState`. The code to implement these enumerations is as follows:

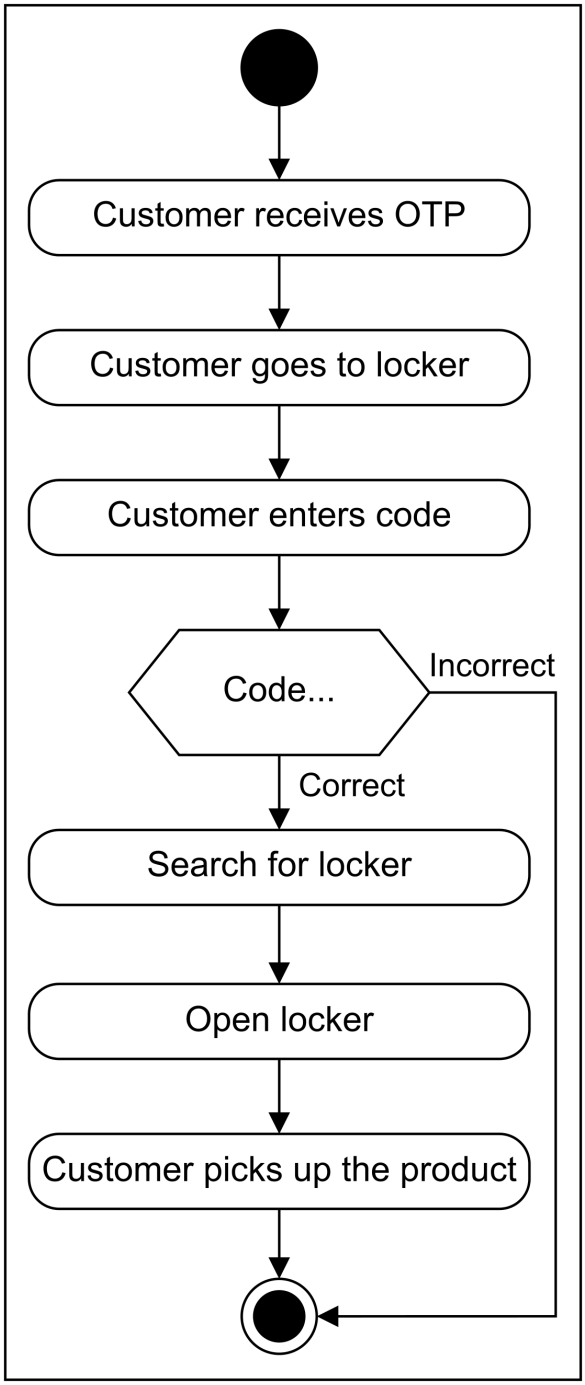


Figure 1.15: The activity diagram for product pickup from the locker

Note: JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Enumerations

```
// definition of enumerations used in the Amazon Locker service
public enum LockerSize { EXTRA_SMALL,
SMALL, MEDIUM,
LARGE, EXTRA_LARGE,
DOUBLE_EXTRA_LARGE }
public enum LockerState { CLOSED,
BOOKED, AVAILABLE
}
```

Item and order

The `Item` class represents the single item, while the `Order` represents the order placed by the customer and can contain a list of items. The definition of these two classes is given below:

Item and Order classes

```
public class Item { private String itemId;
 private int quantity;

 public Item(String itemId, int quantity) { }

 public String getItemId() { }
 public int getQuantity() { }
}

import java.util. List;

public class Order { private String orderId;
private List<Item> items; private String deliveryLocation;
 private String customerId;

 public Order(String orderId, String deliveryLocation, String
    customerId) { }

 public String getOrderId() { }
 public String getDeliveryLocation() { }
 public String getCustomerId() { }
 public List<Item> getItems() { }
 public void addItem(Item item) { }
}
```

Package and LockerPackage

When an order is packed, it is represented by the `Package`, and the package that is contained in the locker is represented by the `LockerPackage` class. The code to implement these classes is shown below:

The Package and LockerPackage classes

```
public class Package { private String packageId;
private double packageSize; private Order order;

 public Package(String packageId, double packageSize, Order order)
     { }

 public String getPackageId() { }
 public double getPackageSize() { }
 public Order getOrder() { }
 public void pack() { }
}

import java.util. Date;

public class LockerPackage extends Package { private int
    codeValidDays;
private String lockerId; private String code;
private Date packageDeliveryTime; private String deliveryPersonId;

public LockerPackage(String packageId, double packageSize, Order
    order, int codeValidDays, String lockerId, String code,
 Date packageDeliveryTime, String deliveryPersonId) { }

 public int getCodeValidDays() { }
 public String getLockerId() { }
 public String getCode() { }
 public Date getPackageDeliveryTime() { }
 public String getDeliveryPersonId() { }

 public boolean isValidCode() { }
 public boolean verifyCode(String code) { }
}
```

Customer and DeliveryPerson

The `Customer` class represents an Amazon user who can place orders, request returns, and receive notifications. The `DeliveryPerson` class models a logistics staff member responsible for delivering packages to lockers, picking up returns, and receiving notifications relevant to returns and deliveries. The implementation of these classes is given below:

Customer and DeliveryPerson

```
public class Customer { private String customerId;
private String name; private String email;
 private String phone;

 public Customer(String customerId, String name, String email,
    String phone) { }

 public void placeOrder(Order order) { }

 public void requestReturn(Order order) { }

 public void receiveNotification(Notification notification) { }
}

public class DeliveryPerson { private String deliveryPersonId;

 public DeliveryPerson(String deliveryPersonId) { }

 public void deliverPackage(LockerPackage pkg, Locker locker) { }

 public void pickupReturn(LockerPackage pkg, Locker locker) { }

 public void receiveReturnNotification(Notification notification)
    { }
}
```

Locker and LockerLocation

The `Locker` is the most important class of the system, and a `LockerLocation` can contain multiple `Locker` instances. The implementation of these classes is given below:

The Locker and LockerLocation classes

```
public class Locker { private String lockerId;
private LockerSize lockerSize; private String locationId;
private LockerState lockerState; private LockerPackage
   currentPackage;

public boolean addPackage(); public boolean removePackage();
}

public class LockerLocation { private String name;
private List<Locker> lockers; private double longitude;
private double latitude; private Date openTime;
 private Date closeTime;

public void addLocker(Locker locker) { // Implement your logic
```

```
    here
 }
}
```

LockerService and Notification

The final class of an Amazon Locker service is the `LockerService` class, which will be a Singleton class, which means that the entire system will have only one instance of this class. The following code provides the definition of the `LockerService` and `Notification` classes used in the Amazon Locker service:

The LockerService and Notification classes

```
public class LockerService { private List<LockerLocation> locations;

 // The LockerService is a Singleton class that ensures it will have only one active instance at a time private static LockerService lockerService = null;

// Created a static method to access the Singleton instance of LockerService class public static LockerService getInstance() { if (lockerService == null) {
lockerService = new LockerService(); }
return lockerService; }

public void addLocation(LockerLocation loc) { // Implement your logic here
 }

public Locker findLockerById(String lockerId) { // Implement your logic here
 }

 // Returns true if the return is approved, false otherwise.
public boolean requestReturn(Order order) { // Implement your logic here
 }

 // Finds (and returns) the first available Locker across all locations.
public Locker requestLocker() { // Implement your logic here
 }

 // Delegates to a specific LockerPackage's verifyCode(...) under the hood.
public boolean verifyOTP(LockerPackage lpkg, String code) { //
```

```
    Implement your logic here
 }
}

public class Notification { private String customerId;
private String orderId; private String lockerId;
 private String code;

public void send(); }
```

7.2 Executable Code: Amazon Locker Service

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Amazon Locker Service system. The system's main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the code.

What does this code show

- **System initialization:** Sets up locations, lockers, a customer, and a delivery person.
- **Scenario 1 (Customer places an order):** The customer places an order, the system packs the order, assigns a locker, the delivery person delivers the package, and the customer is notified with a code.
- **Scenario 2 (Customer picks up the package):** The customer arrives at the locker, enters the code, and successfully retrieves their package.
- **Scenario 3 (Customer initiates a return):** The customer requests a return, the system approves and assigns a locker for the return, sends a return notification, and the delivery person collects the returned package.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Amazon Locker Service

```
import java.util.Calendar;
import java.util.Date;
import java.util.List;

public class Driver {
    public static void main(String[] args) {
        System.out.println(new String(new char[100]).replace('\0',
            '    '));
        System.out.println("\t\t\t\t\tAMAZON LOCKER SERVICE SYSTEM
           ");
```

```java
        System.out.println(new String(new char[100]).replace('\0', '    '));

        // --- System Initialization ---
        System.out.println("         [SETUP] Initializing Locker Service, Locations, and Lockers...\n");
        LockerService lockerService = LockerService.getInstance();
        LockerLocation loc = new LockerLocation("Downtown", 10.5, 20.8, new Date(), new Date());
        Locker locker1 = new Locker("L1", LockerSize.MEDIUM, loc.getName());
        Locker locker2 = new Locker("L2", LockerSize.LARGE, loc.getName());
        loc.addLocker(locker1);
        loc.addLocker(locker2);
        lockerService.addLocation(loc);
        System.out.println("         Added LockerLocation: Downtown with Lockers: [L1, L2]\n");

        // --- Customer and DeliveryPerson Setup ---
        Customer customer = new Customer("CUST1", "Alice", "alice@example.com", "1234567890");
        DeliveryPerson deliveryGuy = new DeliveryPerson("DEL1");

        // === Scenario 1: Customer Places an Order ===
        System.out.println("1         SCENARIO 1: Customer Places an Order");
        System.out.println(new String(new char[100]).replace('\0', '-'));
        Order order = new Order("ORD1", loc.getName(), customer.getCustomerId());
        order.addItem(new Item("ITM1", 2));
        System.out.println("      [Customer] Placing order ORD1 for 2x ITM1.");
        customer.placeOrder(order);

        Package pkg = new Package("PKG1", 2.5, order);
        System.out.println("      [System] Packing the order...");
        pkg.pack();

        System.out.println("      [System] Assigning a locker for package delivery...");
        Locker assignedLocker = lockerService.requestLocker(LockerSize.MEDIUM);
        String otp = "123456";
        LockerPackage lpkg = new LockerPackage("PKG1", 2.5, order, 3, assignedLocker.getLockerId(), otp, new Date(), deliveryGuy.getDeliveryPersonId());
```

```java

        System.out.println("      [DeliveryPerson] Delivering package PKG1 to Locker L1...");
        deliveryGuy.deliverPackage(lpkg, assignedLocker);

        Notification notification = new Notification(customer.getCustomerId(), order.getOrderId(), assignedLocker.getLockerId(), otp);
        System.out.println("      [System] Sending pickup notification to customer...");
        notification.send();
        customer.receiveNotification(notification);

        // === Scenario 2: Customer Picks Up the Package ===
        System.out.println("\n 2     SCENARIO 2: Customer Picks Up the Package");
        System.out.println(new String(new char[100]).replace('\0', '-'));
        System.out.println("      [Customer] Arriving at Locker L1 to pick up package.");
        boolean pickupSuccess = assignedLocker.removePackage(otp);
        if (pickupSuccess) {
            System.out.println("            [Customer] Pickup successful! Package retrieved from locker L1.");
        } else {
            System.out.println("        [Customer] Pickup failed.");
        }

        // === Scenario 3: Customer Initiates a Return ===
        System.out.println("\n 3     SCENARIO 3: Customer Initiates a Return");
        System.out.println(new String(new char[100]).replace('\0', '-'));
        customer.requestReturn(order);
        boolean returnApproved = lockerService.requestReturn(order);

        if (returnApproved) {
            System.out.println("      [System] Return approved! Assigning locker for return...");
            Locker returnLocker = lockerService.requestLocker(LockerSize.MEDIUM);
            String returnOtp = "654321";
            LockerPackage returnPkg = new LockerPackage("PKG1-R", 2.5, order, 3, returnLocker.getLockerId(), returnOtp, new Date(), deliveryGuy.getDeliveryPersonId());
```

```java

            Notification returnNotif = new Notification(customer.getCustomerId(), order.getOrderId(), returnLocker.getLockerId(), returnOtp);
            System.out.println("      [System] Sending return locker details to customer...");
            returnNotif.send();
            customer.receiveNotification(returnNotif);

            System.out.println("      [System] Notifying DeliveryPerson about expected return...");
            deliveryGuy.receiveReturnNotification(returnNotif);

            System.out.println("      [Customer] Placing return package PKG1-R in Locker " + returnLocker.getLockerId() + "...");
            returnLocker.addPackage(returnPkg);
            System.out.println("          [Customer] Return package placed in locker " + returnLocker.getLockerId() + ".");

            System.out.println("      [DeliveryPerson] Arriving to pick up returned package...");
            deliveryGuy.pickupReturn(returnPkg, returnLocker);
        }

        System.out.println("\ n   \t\t\t\t\tSYSTEM DEMO COMPLETE");
        System.out.println(new String(new char[100]).replace('\0', '   '));
    }
}
```

7.3 Wrapping up

We've explored the complete design of an Amazon Locker service in this chapter. We 've examined how an Amazon Locker service design can be visualized using various UML diagrams.

7.4

8 Summary: Designing the Amazon Locker Service

Now that you've completed the Amazon Locker service case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

Here is a list of the primary functional requirements for the Amazon Locker system.

1. Customers can select a preferred locker location for order pickup during checkout.
2. Orders can contain multiple items packaged together based on locker size availability.
3. Locker locations have multiple lockers of various sizes.
4. Only packages within a locker's dimensions are eligible for locker delivery.
5. Customers receive a unique code to open the locker when a package is delivered.
6. Packages are held in a locker for a maximum of three days.
7. Package pickup must occur within the three-day window and the locker location's operating hours.
8. After three days, uncollected packages are removed, the locker is released, and a refund is initiated.
9. Each locker can only be assigned to one customer/package at a time.
10. The locker's access code is invalidated once a package is collected.
11. Customers can return eligible items by selecting a nearby locker location and receiving a new code for an available locker.
12. The logistics team picks up returned items using a unique code, after which the customer is notified and refunded.

8.2 System actors

The following are the primary and secondary actors who interact with the system.

- **Customer:** Selects locker locations, picks up packages using a code, and initiates returns by dropping off packages.
- **Delivery person:** Delivers packages to lockers for customer pickup and collects returned items from lockers.

8.3 Important classes and relationships

Below are the system design's most important classes, descriptions, and key relationships.

Class	Description	Important Relationships
Customer	Represents the user who places orders and initiates returns.	Association with Order and Notification.
DeliveryPerson	Represents the logistics staff who deliver packages and collect returns.	Association with LockerPackage and Notification.
Order	Models a transaction initiated by a customer, containing items and a delivery location.	Composition of Item, Association with Notification.
Package	Encapsulates the physical bundle for delivery or return, with a specific size.	Association with Order.
LockerPackage	A specialized package with locker-specific details like an access code and validity period.	Inheritance from Package, Composition of Locker.
Locker	Represents a single physical locker compartment with a specific size and status.	Aggregation by LockerLocation.
LockerLocation	Models a physical site containing multiple lockers, with operating hours and a location.	Composition of Locker, Aggregation by LockerService.
LockerService	The main class that manages all locker locations and orchestrates the core logic.	Aggregation of LockerLocation.
Notification	Manages user communication (e.g., delivery, return, overdue notices).	Association with Customer and DeliveryPerson.

8.4 Design highlights

This section covers the overall design approach, the core principles applied, and the key design patterns used in the system.

Design approach

The system was designed using a bottom-up approach. We started by identifying core entities like `Locker`, `Package`, and `Order`. We then built upon these to model more complex operations such as locker assignment, package handling, and secure access, ensuring the design supports concurrency, scalability, and maintainability.

Design principles

The design of the Amazon Locker system implicitly incorporates the SOLID principles to ensure it is robust and maintainable.

- **Single Responsibility principle (SRP):** Each class has a distinct responsibility. For example, the `Locker` class manages the state of a single locker, while the `Notification` class is solely responsible for sending communications to users.
- **Open/Closed principle (OCP):** The system is open to extension but closed for modification. For instance, the `LockerPackage` class extends the `Package` class, adding new functionality for locker-specific interactions without altering the base `Package` class.
- **Liskov Substitution principle (LSP):** Subtypes can be substituted for their base types. A `LockerPackage` object can be used wherever a `Package` object is expected, ensuring consistent behavior.
- **Interface Segregation principle (ISP):** The system's classes and interactions are designed to be specific. For example, the `Customer` and `DeliveryPerson` interact with the `LockerService` through focused methods (`requestReturn`, `deliverPackage`) rather than a single, large interface.
- **Dependency Inversion principle (DIP):** High-level modules do not depend on low-level modules; both depend on abstractions. The `LockerService` depends on abstract concepts of lockers and packages, not their concrete implementations, allowing for flexibility.

Design patterns

The following design patterns were identified as beneficial for structuring the Amazon Locker system.

- **Strategy pattern:** This pattern encapsulates different algorithms for tasks like locker assignment (e.g., finding the closest, best-fit, or least-used locker) and OTP generation, allowing these strategies to be swapped easily.
- **Repository pattern:** This pattern separates the data access and business logic. For example, a `LockerRepository` would handle all database operations for `Locker` objects, simplifying the `LockerService`.

8.5 Object interactions

As typical interaction flows illustrate, the system's dynamic behavior is driven by collaborations between key objects to accomplish tasks.

- **Package return:** A `Customer` initiates a return via the `LockerService`. The service verifies the request, finds an available `Locker`, and uses the `Notification` object to send the customer an access code. The customer uses this code to place the package in the `Locker`, and the `LockerService` updates the locker's state.
- **Package pickup:** A `DeliveryPerson` places a package in a `Locker`. The `LockerService` records this, generates a code, and sends a `Notification` to the `Customer`. The `Customer` uses the code at the `LockerLocation`, which the `LockerService` validates to open the correct `Locker`.
- **Code validation:** For any locker access, the `LockerService` receives a code, validates it against the `LockerPackage` data, and if correct, interacts with the specific `Locker` object to unlock it.

8.6 System workflow

The following are a few primary end-to-end scenarios that highlight the system's life cycle from a user's perspective:

- **Order and pickup workflow:** A customer places an order and selects a `LockerLocation`. A `DeliveryPerson` delivers the package to an assigned locker. The Amazon Locker System sends the customer a notification with a pickup code. The customer goes to the locker, enters the code, the system validates it, opens the locker, and the customer retrieves their product.
- **Return workflow:** A customer submits a return request through the system. The system approves it and finds a suitable locker. It then sends a notification with a new code to the customer. The customer goes to the location, enters the code to open the locker, and places the return package inside. A `DeliveryPerson` is later notified to collect the returned item.

9 Key Insights and Tips: Designing the Amazon Locker Service

Congratulations on completing the case study on designing the Amazon Locker service. Now , you will be presented with a summary of key insights to help you solidify your understanding. This lesson will cover common mistakes to avoid, essential interview tips, a short quiz to test your knowledge, and a list of related case studies for further practice.

9.1 Common mistakes

Avoiding these common mistakes will help you develop a more efficient and scalable Amazon Locker service:

- **Forgetting package eligibility checks:** Designing the system without a mechanism to ensure a package can fit into an assigned locker. Always implement a check to validate that the package size is less than or equal to the locker size before assignment.

- **Ignoring the time limit for package pickup:** Failing to account for packages not picked up by the customer. The system should include a rule, such as a three-day pickup window, after which the package is removed and a refund is processed.

- **Hardcoding the pickup code:** Using a static or easily predictable code for locker access. Implement a dynamic, secure 6-digit code generation system for each delivery to ensure only the correct customer can access the locker.

- **Lacking a notification system:** Not including a mechanism to inform customers about their delivery. A robust notification component is crucial for sending delivery confirmations, access codes, and pickup reminders.

- **Failing to consider concurrency:** Not addressing the possibility of multiple customers trying to book the same locker simultaneously. Use proper database transactions or locking mechanisms to prevent double-booking and ensure data integrity.

9.2 Interview tips

Here are some essential tips to help you prepare for an interview where you're asked to design an Amazon Locker service:

Tip	Explanation
Justify your choice of design patterns.	When you decide to use a pattern like the Strategy or Repository pattern, explain your reasoning. For example, explain that the Strategy pattern is ideal for implementing different locker assignment algorithms (e.g., based on size, location).
Emphasize scalability in your design.	Discuss how your system would handle increased users, lockers, and locations. Mentioning a LockerService class that manages a list of LockerLocation objects shows you are thinking about scalability.
Discuss the data model and enumerations.	Explain the enumerations you would use, like LockerState and LockerSize. This demonstrates attention to detail and an understanding of managing states and types effectively within the system.
Address the package validity period.	Clearly state the business rule for how long a package can remain in a locker. Explaining the three-day limit and the subsequent refund process shows you can translate business requirements into system logic.
Talk about the physical and system security.	Mention the importance of a secure code for locker access. Discussing how the system validates the code and ensures a locker is closed after pickup demonstrates a holistic understanding of the problem.

9.3 Test your understanding

Take a short quiz to evaluate your understanding of the Amazon Locker service design.

Amazon Locker Service

Question 1: What is the primary mechanism for a customer to access their package in a locker?

- A physical key
- **[CORRECT]** A 6-digit code
  Explanation: The system generates a unique 6-digit code for each delivery, which is sent to the customer to open the locker.
- A credit card swipe
- A mobile app's Bluetooth signal

Question 2: Which class in the design is responsible for managing a collection of individual lockers at a specific address?

- `Order`
- `LockerService`
- `Package`
- **[CORRECT]** `LockerLocation`
  Explanation: The `LockerLocation` class contains a list of `Locker` objects and information like address and operating hours.

Question 3: What is the purpose of the Strategy design pattern in this system?

- **[CORRECT]** To manage different locker assignment and OTP generation algorithms.
  Explanation: The Strategy pattern is suggested for handling various strategies like locker assignment, filtration, and OTP generation.
- To create different types of notifications.
- To ensure only one instance of the `LockerService` is created.
- To define the skeleton of an algorithm in a base class.

Question 4: What kind of relationship does the `LockerPackage` class have with the `Package` class?

- Association
- Aggregation
- Composition
- **[CORRECT]** Inheritance
  Explanation: The `LockerPackage` class extends the `Package` class, inheriting its properties and adding specific attributes like `codeValidDays`.

Question 5: Who are the two primary actors interacting with the Amazon Locker system?

- Customer and system
- **[CORRECT]** Customer and delivery guy
  Explanation: The use case diagram identifies the Customer (for pickup/returns) and the Delivery Guy (for drop-off/returns) as the primary actors.
- Delivery guy and system
- Amazon admin and customer

9.4 Related case studies

Explore these related case studies to deepen your understanding of the design principles applied in the Amazon Locker Service:

- **Designing a parking lot:** This case study also involves managing a finite set of spaces (parking spots vs. lockers) and assigning them to users based on type and availability. Both systems require robust state management and concurrency control.
- **Designing a hotel management system:** This involves booking rooms, which is analogous to booking lockers. Both systems manage inventory (rooms/lockers) and handle reservations, check-ins (package drop-off), and check-outs (package pickup).
- **Designing a vending machine:** This case study is about managing inventory in a vending machine and handling user selections and transactions. It shares the core concept of a self-service automated system with a finite set of items/spaces.
- **Designing a warehouse management system:** This is a larger-scale version of the locker problem, dealing with storing, tracking, and retrieving many items in a warehouse. It involves complex logic for space allocation and inventory management.
- **Designing a bike sharing system:** This system manages a fleet of bikes that can be picked up and dropped off at various locations (docks). It requires tracking the state and location of each bike, similar to tracking the state of each locker.